





- Node.js is an open-source, cross-platform javascript runtime environment built on Chrome's V8 Javascript engine.
- A runtime environment is where your program will be executed.
- Before Node.js we can only execute javascript in runtime environment of a browser like chrome, Edge and Firefox etc.
- Javascript engine is a program whose responsibility is to execute javascript code.
- All modern browsers come with their own version of the javascript engine but the most popular one is Google's V8 Engine(V8 is a free and open-source javascript engine written in C++ and developed by Google that powers Google chrome..
- Node.js is an open-source, cross-platform, back-end javascript runtime environment that runs on the V8 engine and executes javascript code outside a web browser.

- So using Node.js we can execute javascript code outside a web browser.
- <https://nodejs.org/>

Why Node.js?/ Uses-Node.js

- Node.js is very fast in code execution.
- Node.js can generate dynamic page content.
- Node.js can create,open,read,write,delete and close files on the server.
- Node.js can collect form data..
- Node.js can add,delete,modify data in your database.

Modules

Module in Node.js is a simple or complex functionality organized in single or multiple javascript files or library which can be reused throughout the Node.js application.

Node.js includes three types of modules:

- 1.Core Modules/Built in Modules
- 2.Local Modules
- 3.NPM/Third party modules

Core Modules/Built in Modules

Node.js has many built-in modules that are part of the platform and comes with Node.js installation.

To include a module , use the `require()` function with the name of the module: Syntax:

```
var/const module=require('module_name');
```

The following table lists some of the important core modules in Node.js.

http	http module includes classes, methods and events to create Node.js http server.
url	url module includes methods for URL resolution and parsing.
querystring	querystring module includes methods to deal with query string.
path	path module includes methods to deal with file paths.
fs	fs module includes classes, methods, and events to work with file I/O.
util	util module includes utility functions useful for programmers.
os	provides information about the operating system.

Core Modules : HTTP Module

HTTP module allows Node.js to transfer data over the Hyper text transfer protocol(HTTP).

Node.js as a Web Server

The HTTP module can create an HTTP server that listens to server ports and gives a response back to the client

Core Modules :OS Module

1. `os.arch()`--This method is used to fetch the operating system CPU architecture.
2. `os.cpus()`--This method is used to fetch an array of objects containing information about each cpu/core installed: model, speed (in MHz), and times (an object containing the number of milliseconds the cpu/core spent in
3. `os.freemem()`--This methods returns the amount of free system memory in bytes.
4. `os.homedir()`---This method returns the home directory of the current user.
5. `os.hostname()`---This method is used to returns the hostname of the operating system.
6. `os.platform()`--This method returns the operating system platform of the running computer i.e. 'darwin', 'win32', 'linux', etc.
7. `os.release()`---This method returns the operating system release.

8. `os.tmpdir()`--This method returns the operating system's default directory for temporary files.
9. `os.totalmem()`---This method returns the total amount of system memory in bytes.
10. `os.type()`--This method returns the operating system name. For example 'linux' on linux, 'darwin' on mac os and 'windows_nt' on windows.

Core Modules : File System-FS-Module

The Node.js file system module allows you to work with the file system on your computer.

Common uses:

- Read files: method is used to read files on your computer.
`const fs = require('fs');`
`fs.readFile()`
- Create files
`fs.writeFile()`, `fs.open()`
- Update files
`fs.appendFile()`

- Delete files
fs.unlink()
- Rename files
fs.rename()

Core Modules : Node.js Path Module

Path module allows you to interact with file paths easily. Path module has many useful properties and methods to access and manipulate paths in the file system.

Example:

path.extname(p).It returns the extension of the path, from the last '.' to end of string in the last portion of the path.

path.dirname(p).It return the directory name of a path. It is similar to the unix dirname command.

Local Modules

We can create our own modules and we can use it in our project. These modules are called local modules. Using the require function we can use these modules in our projects.

NPM (Node Package Manager)

NPM (Node Package Manager) is the default package manager for Node.js and is written entirely in Javascript.

NPM manages all the packages and modules for Node.js and consists of command line client npm. It gets installed into the system with installation of Node.js. The required packages and modules in Node project are installed using NPM.

Installation / Uninstallation –

Third-Party Modules

Installing Modules using NPM

```
npm install <module-name>
```

After installation:

- The module is added to the **node_modules/** directory.
- An entry is created under **dependencies** in package.json.

Uninstalling Modules using NPM

`npm uninstall <module-name>`

After uninstallation:

- The module is removed from the `node_modules/` directory.
- The corresponding entry is removed from dependencies in `package.json`.

Verify Installed Modules

`npm ls`

Node.js Global Objects

Node.js global objects are the objects that are available in all modules.

These are built in objects/functions that are part of javascript and can be used in our applications without importing any particular module.so You don't need to include these objects in your application; rather they can be used directly.

A list of Node.js global objects are given below:

- `_dirname`

- `_filename`
- `Console`
- `setInterval(callback, delay[, arg][, ...])`
- `setTimeout(callback, delay[, arg][, ...])`
- `clearInterval(intervalObject)`
- `clearTimeout(timeoutObject)`
- `require("")`